

Final project script 3

Step 3. Perform k-means clustering of features for an unsupervised learning coin-type classifier and visualize the results

- Apply the k-means algorithm with $k=3$ on feature matrix **D** to obtain classification **cls_init**. Make sure `rng(0)` is called prior to run different result since k-means uses random numbers in its optimization. Clustering should result in distinct classification of the coins.
- Map the labels assigned by k-means to each class to a numeric label for each coin of 1, 2, and 3 for dimes, nickels, and quarters. Store the average area of the components in each of the 3 classes in the $[3 \times 1]$ vector **class_ave_object_size** (the size of each component stored in **component_size**). The class corresponding to dimes should have the smallest average size, nickels the next smallest, and quarters the largest. Labels in **cls_init** to $[1,2,3]$ for smallest to largest coin ordering, find the sorting indices **classmap** needed to sort **class_ave_object_size** using **sort** function. **classmap(1)** contains the label of the smallest (dime) class, **classmap(2)** the middle (nickel) class, and **classmap(3)** the largest (quarter) class. Create an $[N \times 1]$ classification vector **cls** such that **cls(i)** equals 1 if **cls_init(i)** equals **classmap(1)**, **cls(i)** equals 2 if **cls_init(i)** equals **classmap(2)**, and **cls(i)** equals **classmap(3)**. This converts the k-means labels in **cls_init** to be $[1,2,3]$ for dimes, nickels, and quarters in **cls**. We now have the results in a way that we can easily see if they are correct.
- In a separate MATLAB grader problem, you are asked to create function `AddCoinToPlotAndCount`. **Complete that problem now.** Embed the resulting function in the bottom of the script with the other Helper Functions.
- For each classified centroid, use the `AddCoinToPlotAndCount` function to draw a circle on the image display for each localized centroid and diameter approximately matching that of the coin.
- The function also outputs the value of the coin being plotted. Sum this result across all coins and store the result in **totcount**, which is a $[3 \times 1]$ vector.

Script ?

[Save](#)[Reset](#)[MATLAB Documentation \(https://www.mathworks.com/help/\)](https://www.mathworks.com/help/)

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42

```

43 % Define the filter size we will use in step 2:
44 filtsize = 85;
45
46 % Creating test image 'im' by splicing together two built in images.
47 % Also zero-padding (adding zeros around the border) with half the
48 % filter size (filtsize) we will use so that the filter could be
49 % centered on any actual image pixel, including those near the border.
50 % 'coins.png' contains bright nickels and dimes on a dark background
51 % 'eight.tif' contains dark quarters on a bright background, so we invert it
52 % to match 'coins.png'
53 im1 = imread('coins.png');
54 [r,c] = size(im1);
55 im2 = imread('eight.tif');
56 [r2,c2] = size(im2);
57 filtsizeh = floor(filtsize/2);
58 im = zeros(r+r2+filtsize,c+filtsize);
59 im(filtsizeh+1:filtsizeh+r+r2,filtsizeh+1:filtsizeh+c) = [im1;255-im2(:,1:c)];
60 [r,c] = size(im);
61 imagesc(im);colormap(gray);title('test image');axis equal;
62
63 % Initializing assessed/displayed variables as empty so that code is executable
64 msk=[]; msk_dil=[]; msk_dil_erd=[]; centroid=[]; component_size=[];
65
66 %%%% 1. Localize the centroid of each coin
67 % Otsu threshold
68 msk = OtsuThreshold(im);
69 figure; imagesc(msk); colormap(gray); title('Otsu'); axis equal;
70
71 % Dilate 9x9
72 msk_dil = imdilate(msk,ones(9,9));
73 figure; imagesc(msk_dil); colormap(gray); title('Dilated'); axis equal;
74
75 % Erode 23x23
76 msk_dil_erd = imerode(msk_dil,ones(23,23));
77 figure; imagesc(msk_dil_erd); colormap(gray); title('Eroded'); axis equal;
78
79
80 % Connected components to get centroids of coins:
81 cc = bwconncomp(msk_dil_erd);
82 props_struct = regionprops(cc);
83 centroid = zeros(length(props_struct),2);
84 component_size = zeros(length(props_struct),1);
85 for i=1:length(props_struct)
86     centroid(i,:) = round(props_struct(i).Centroid);
87     component_size(i) = props_struct(i).Area;
88 end
89
90
91 %%%% 2. Measure features for each coin using a bank of matching filters
92 % make matching filters to create features
93 % Define diameters to use for filters
94 dimediameter = 31;
95 quarterdiameter = 51;
96 nickeldiameter = 41;
97
98 % Initialize assessed variable D
99 D=[]; nickelfilter = []; dimefilter = []; quarterfilter = [];
100
101 % Use the MakeCircleMatchingFilter function to create matching filters for dimes, nickels, and quarters
102 % (This is in a separate Matlab grader problem. Save your work,
103 % complete the corresponding grader problem and embed the solution
104 % in the helper function list below.)
105 nickelfilter = MakeCircleMatchingFilter(nickeldiameter,filtsize);
106 dimefilter = MakeCircleMatchingFilter(dimediameter,filtsize);
107 quarterfilter = MakeCircleMatchingFilter(quarterdiameter,filtsize);
108
109 figure;
110 subplot(1,3,1); imagesc(dimefilter); colormap(gray); title('dime filter'); axis tight equal;
111 subplot(1,3,2); imagesc(nickelfilter); colormap(gray); title('nickel filter'); axis tight equal;
112 subplot(1,3,3); imagesc(quarterfilter); colormap(gray); title('quarter filter'); axis tight equal;
113
114 % Evaluate each of the 3 matching filters on each coin to serve as 3 feature measurements
115 D = zeros(length(centroid),3);

```

```

115 D = zeros(length(centroid),3),
116 for i=1:length(centroid)
117     D(i,1) = corr(dimefilter(:),reshape(msk_dil_erd(centroid(i,2)-filtsizeh:...,
118         centroid(i,2)+filtsizeh,centroid(i,1)-filtsizeh:centroid(i,1)+filtsizeh),[filtsize*filtsize,1]));
119     D(i,2) = corr(nickelfilter(:),reshape(msk_dil_erd(centroid(i,2)-filtsizeh:...,
120         centroid(i,2)+filtsizeh,centroid(i,1)-filtsizeh:centroid(i,1)+filtsizeh),[filtsize*filtsize,1]));
121     D(i,3) = corr(quarterfilter(:),reshape(msk_dil_erd(centroid(i,2)-filtsizeh:...,
122         centroid(i,2)+filtsizeh,centroid(i,1)-filtsizeh:centroid(i,1)+filtsizeh),[filtsize*filtsize,1]));
123 end
124
125 figure;
126 subplot(1,3,1); imagesc(dimefilter); colormap(gray); title('dime filter'); axis tight equal;
127 subplot(1,3,2); imagesc(nickelfilter); colormap(gray); title('nickel filter'); axis tight equal;
128 subplot(1,3,3); imagesc(quarterfilter); colormap(gray); title('quarter filter'); axis tight equal;
129
130 %%%% 3. Perform k-means clustering of features for unsupervised learning classifier
131 rng(0);
132 cls_init=[]; cls=[]; totcount=[];
133 [cls_init,~] = kmeans(D,3);
134
135 cls_init
136 % relabel centroid classes based on average size of the objects in each class. smallest will be dime, next
137 class_ave_object_size = zeros(3,1);
138 for i=1:3
139     class_ave_object_size(i) = mean(component_size(cls_init==i));
140 end
141
142 [~,clsmap] = sort(class_ave_object_size);
143 %class_ave_object_size
144 %clsmap
145
146 for i=1:size(centroid,1)
147     %cls(i) = clsmap(cls_init(i));
148     for j=1:3
149         if cls_init(i) == clsmap(j)
150             cls(i) = j;
151         end
152     end
153 end
154
155 cls = cls';
156
157 cls
158 % Visualize the result
159 figure; imagesc(im);colormap(gray);title('test image');hold on;axis equal;
160
161 % plot circles around each coin with different color/diameter unique to each type and count the change
162 totcount = 0;
163 for i=1:size(centroid,1)
164     x = centroid(i,1);
165     y = centroid(i,2);
166     [coinvalue,~,~,~] = AddCoinToPlotAndCount(x,y,cls(i));
167     totcount = totcount + coinvalue;
168 end
169
170 title([num2str(totcount), ' cents'])
171
172
173 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Helper Functions %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
174 function [coinvalue,x_plot,y_plot,col] = AddCoinToPlotAndCount(x,y,cls)
175 % initialize radians for defining x_plot and y_plot using cos and sin functions
176 rads = 0:2*pi/32:2*pi;
177 % initialize parameters for radius and color of circle for each type of coin
178 r = 22;
179 col = 'r';

```

▶ Run Script



Assessment: All Tests Passed

✓ Is cls_init correct?

✓ Is cls correct?

✓ Is totcount correct?

Output

```
cls_init =
```

```
3  
1  
2  
3  
2  
1  
3  
3  
3  
3  
1  
3  
1  
1  
2  
2
```

```
cls =
```

```
2  
3  
1  
2  
1  
3  
2  
2  
2  
2  
3  
2  
3  
1  
1
```

